

# 03-01-K-NN-Lab

January 19, 2026

<div style='float:left;'><h1>CPSC 4300/6300: Applied Data Science</h1></div>



## 1 Week 3 | Lab 3: k-NN Regression

Clemson University Instructor(s): Tim Ransom

---

### 1.1 Learning objectives

- Implement the k-Nearest Neighbors algorithm for regression using scikit-learn.
  - Evaluate the performance of a k-NN regression model by calculating the R-squared score on training and test datasets.
  - Interpret the impact of different values of 'k' on model performance, including bias-variance trade-off.
- 

### 1.2 About

**This Lab exercise is focused on analyzing COVID-19 case data from selected South Eastern U.S. states using K-NN Regression.**

**This Lab involves: Data Loading and Preprocessing:** Loading COVID-19 data from a CSV file, filtering out invalid rows (such as those with zero or negative cases), and assigning unique day indices to track time progression.

**Data Filtering:** Creating a dataset specifically for South Eastern states (South Carolina, North Carolina, Georgia, Florida, Tennessee, Mississippi, and Alabama) to narrow the focus of the analysis.

**Data Splitting:** Splitting the filtered data into training and test datasets with a 70/30 split to evaluate model performance on unseen data.

**Model Training with k-Nearest Neighbors:** Training multiple kNN regression models with different values of k to observe the effects of different neighbor sizes. These models will be used to predict daily COVID-19 cases.

**Model Evaluation:** Calculating  $R^2$  scores for both the training and test datasets for each value of k to quantitatively evaluate the performance of the models. A plot of  $R^2$  values to visualize how the model's performance changes as k varies.

**Data Visualization:** Various visualizations including:

1. A plot of the raw training data.
2. Predicted vs actual case plots for each kNN model, highlighting how well each model fits the data.
3. A plot of the train and test  $R^2$  values as a function of  $k$  to help understand how the choice of  $k$  affects model performance.

Overall, the exercise aims to demonstrate how kNN regression models can be applied to real-world data for time-based prediction. It also explores the trade-offs between different model complexities by varying  $k$  and evaluating the impact on model performance, using both visualization and statistical metrics.

```
[ ]: """ RUN THIS CELL TO GET THE RIGHT FORMATTING """
import requests
from IPython.core.display import HTML
css_file = 'https://raw.githubusercontent.com/bsethwalker/clemson-cs4300/main/
↪css/cpsc6300.css'
styles = requests.get(css_file).text
HTML(styles)
```

```
[ ]: # Import necessary libraries for numerical operations, data manipulation,
    ↪modeling, and visualization
import numpy as np
import pandas as pd

from sklearn.metrics import r2_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import statsmodels.api as sm
from statsmodels.api import OLS

import matplotlib.pyplot as plt
import seaborn as sns
sns.set() # set theme for seaborn visualization
# set seed value

from matplotlibcheck.base import PlotTester
from matplotlib.patches import PathPatch
```

Exercise 1:

- Load the data from `data/us-states.csv` into a DataFrame called `us_covid`

```
[ ]: """Write your code for exercise-1 here:"""

# your code here
raise NotImplementedError
```

```
[ ]:
```

### Exercise 2:

- Create a column called `day` where the column contains the day number. For example, 0 is the first date of data, 1 is the second, etc. Each unique date will get a unique value in the day column, so if 2020-01-24 is in the date column twice, the corresponding day value will be present twice as well.

```
[ ]: """Write your code for exercise-2 here:"""
```

```
# your code here  
raise NotImplementedError
```

```
[ ]:
```

```
[ ]: # you should be able to visualize your data using this line  
plt.scatter(us_covid.day, us_covid.cases)
```

### Exercise 3:

- Remove any rows where the number of cases is less than or equal to zero.

```
[ ]: # Shape of dataframe before removing rows where number of cases is less than or  
↳ equal to zero.  
print(us_covid.shape)  
# Number of rows where the number of cases is less than or equal to zero.  
(us_covid.cases <= 0).sum()
```

```
[ ]: # your code here  
raise NotImplementedError
```

```
[ ]: # Shape of dataframe after removing rows where number of cases is less than or  
↳ equal to zero.  
us_covid.shape
```

```
[ ]:
```

### Exercise 4:

- Create a new DataFrame, called `se_covid`, which contains data from states in the South Eastern U.S. (South Carolina, North Carolina, Georgia, Florida, Tennessee, Mississippi and Alabama)

The `set` function in Python is used to create a mathematical set object, which is an unordered collection of unique elements (no repeats, only one of anything).

```
[ ]: us_covid.state.isin(set(['South Carolina']))
```

In above code `us_covid.state` selects the state column from the `us_covid` DataFrame, which contains the state names for each row.

`set(['South Carolina'])` creates a set containing a single element 'South Carolina'.

The `.isin()` function checks if each value in the state column is present in the provided set. In this case, it will check if each state's name in the `us_covid` DataFrame is 'South Carolina'.

The result of this code is a boolean Series (a column of True or False values) of the same length as the `us_covid` DataFrame. Each value will be:

True if the corresponding state value is South Carolina False otherwise

```
[ ]: south_east_states = set(['South Carolina', 'North Carolina', 'Georgia',  
    ↪ 'Florida', 'Tennessee', 'Mississippi', 'Alabama'])
```

Above code creates a set that contains the names of seven states: South Carolina, North Carolina, Georgia, Florida, Tennessee, Mississippi, and Alabama.

This set is assigned to the variable `south_east_states`.

```
[ ]: """Write your code for exercise-4 here:"""  
  
    # your code here  
    raise NotImplementedError
```

```
[ ]:
```

```
[ ]: # print the number of rows and columns in the se_covid DataFrame  
    se_covid.shape
```

```
[ ]: # print first few elements of se_covid dataframe  
    se_covid.head()
```

Exercise 5:

- Split `se_covid` into training and test sets. Namely - `train_data` and `test_data` with an 70/30 split using `random_state = 42`.

```
[ ]: """Write your code for exercise-5 here:"""  
  
    # your code here  
    raise NotImplementedError  
    print(train_data)
```

```
[ ]:
```

```
[ ]: # you should be able to see the first section of your data with this piece of  
    ↪ code  
    train_data.head()
```

Exercise 6:

- Plot the training data.

- Create a scatter plot of COVID-19 daily case count in South Eastern U.S. States using `train_data` dataset where x-axis (`day`) represents the day since the start of the data and y-axis (`cases`) represents number of COVID-19 cases recorded for each day.
  1. To create the scatter plot, you need to set up the Figure and Axes objects using `plt.subplots()`.
    - These two objects (`fig` and `ax`) will allow you to control various properties of the figure and plot.
    - `fig` is the Figure object: It serves as the overall container for the plot.
    - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.

Example code: `fig, ax = plt.subplots(figsize=(10, 6))`
  2. Customize the Plot:
    - You need to set the title of the plot, the labels for the x-axis and y-axis, and format the ticks on the x-axis for better readability.

```
[ ]: """Write your code for exercise-6 here: """
```

```
# your code here
raise NotImplementedError
```

```
[ ]:
```

### 1.2.1 Using a `KNeighborsRegressor` model from `sklearn` to perform a regression task.

```
[ ]: # Create a k-NN regression model with 50 neighbors
model = KNeighborsRegressor(n_neighbors=50)
```

```
[ ]: # note the formatting of outputting a model in jupyter, we do not typically
      ↪ print the model like this and instead tell our audience metrics about it
      ↪ such as loss and accuracy
model.fit(
    train_data[['day']], # Input needs to be a Pandas DataFrame or a Numpy array
    train_data[['cases']])
```

### 1.2.2 `KNeighborsRegressor(n_neighbors=50)`:

The above line initializes a K-nearest neighbors (KNN) regression model.

`KNeighborsRegressor` is an algorithm from `scikit-learn`'s `neighbors` module, which is used for regression tasks.

`n_neighbors=50` specifies that the model will use the 50 nearest data points to make predictions. This means that, when making a prediction for a given day, the algorithm will find the 50 closest neighbors (in terms of feature space) and compute the average of their corresponding target values to predict the value for the input.

### 1.2.3 `model.fit(train_data[['day']], train_data[['cases']])`:

The above line trains the KNN regression model on the training data.

`fit()` is the method used to train the model.

It takes two arguments:

**Input Features** (`train_data[['day']]`): The input features should be provided as a DataFrame or a Numpy array. In this case, `train_data[['day']]` is used, which selects the day column from the `train_data` DataFrame. Notice that it is written as `[['day']]`, which means it is passed as a 2D DataFrame, rather than a 1D series. This is important, as the model expects a 2D structure for input features.

**Target Variable** (`train_data[['cases']]`): This is the column that the model is trying to predict. `train_data[['cases']]` is used, again as a 2D DataFrame, which contains the target values that correspond to the feature input. These are the COVID-19 case counts for each day. The model will learn the relationship between the day (as input) and cases (as output) by finding and storing relevant data points for future predictions.

Exercise 7:

- Create a dictionary named - `KNN_models` containing k-NN Regression models where the dictionary key is the value of k and the value is the fitted k-NN model with the corresponding value of k.
- Use the following values of k: 1, 10, 75, 250, 500, 750, 1000, and the number of examples in the training set.

```
[ ]: # Create two k-NN regression models, one with 1 neighbor and another with 10
      ↪neighbors
knn1 = KNeighborsRegressor(n_neighbors=1)
knn10 = KNeighborsRegressor(n_neighbors=10)
```

```
[ ]: # Dictionary to store k-NN models with different neighbor values
example_dictionary = {1: KNeighborsRegressor(n_neighbors=1), 10:
      ↪KNeighborsRegressor(n_neighbors=10)}
example_dictionary
```

```
[ ]: # defining k values
k_values = [1, 10, 75, 250, 500, 750, 1000, train_data.shape[0]]
k_values
```

```
[ ]: """Write your code for exercise-7 here: """

KNN_models = {}

# your code here
raise NotImplementedError
```

```
[ ]:
```

Exercise 8:

- Create two dictionaries containing the predictions on both the train and test datasets.

- Name these dictionaries `knn_predicted_pickups_train` and `knn_predicted_pickups_test` respectively.

```
[ ]: """Write your code for exercise-8 here:"""
```

```
knn_predicted_pickups_train = {}
knn_predicted_pickups_test = {}

# your code here
raise NotImplementedError
```

```
[ ]:
```

Exercise 9:

- Plot the training data of each model, as well as the model predictions.
1. Define the function `plot_knn_prediction` to visualize the predicted vs actual values:
    - The function accepts `ax` (axis to plot on), `dataset` (data to plot), `predictions` (predicted values), `k` (number of neighbors), and `dataset_name` (name of the dataset, e.g., Training or Test).
    - Plot the actual data using `ax.plot` with `'.'` as the point marker, which allows for clear visualization of actual data points.
    - Plot the predicted values using `ax.plot` with `'*'` to clearly differentiate from actual data.
    - Set the plot title, x-label, y-label, and include a legend to understand the plotted data.
  2. Create the figure for visualizing predictions:
    - Use `plt.subplots()` to create subplots with `nrows` equal to the number of `k_values` and 2 columns (one for training data, one for test data).
    - Set the figure size to (16, 28) for clarity, and add a general title for the whole figure using `fig.suptitle()`.
  3. Iterate through different values of `k` using a loop:
    - For each value of `k` in `k_values`, use `plot_knn_prediction()` function to plot the training data on the left column and test data on the right column.
    - `axes[i][0]` is used to plot on the training subplot and `axes[i][1]` for the test subplot.
    - `knn_predicted_pickups_train[k]` and `knn_predicted_pickups_test[k]` hold the predicted values for training and test datasets respectively.
  4. Adjust the figure layout using `fig.tight_layout()`:
    - This ensures that subplots are arranged nicely, and labels/titles don't overlap.

```
[ ]: """Write your code for exercise-9 here:"""
```

```
# fill in this function skeleton with your code
def plot_knn_prediction():
    pass # remove this pass when the function is implemented

# your code here
raise NotImplementedError
```

```
[ ]:
```

Exercise 10.1:

- What happens as  $k$  increases?
  1. the prediction converges
  2. the prediction diverges
  3. undefined behavior

Write your selection to a variable named **answer** in the cell below.

```
[ ]: # your code here
      raise NotImplementedError
```

```
[ ]:
```

Exercise 10.2:

- What prediction do we get when  $k$  is equal to the number of training points?
  1. the average of the data set
  2. the median of the data set
  3. the mode of the data set
  4. the standard deviation of the data set

Write your selection to a variable named **answer** in the cell below.

```
[ ]: # your code here
      raise NotImplementedError
```

```
[ ]:
```

Exercise 11:

- Calculate and report the train and test R2 values for each model.
- Store the values in dictionary named: `train_r2_scores = {}` and `test_r2_scores = {}`

```
[ ]: """Write your code for exercise-11 here: """
```

```
# your code here
raise NotImplementedError
```

```
[ ]:
```

```
[ ]: # This format makes the display much more readable
knn_r2_df = pd.DataFrame(data = {"k" : tuple(train_r2_scores.keys()),
                                "Train R^2" : tuple(train_r2_scores.
↵values()),
                                "Test R^2" : tuple(test_r2_scores.
↵values())})

knn_r2_df
```

### Exercise 12:

- Create a line plot of the train/test  $R^2$  scores as a function of  $k$ .
- Plot the  $R^2$  scores for training and testing datasets
- Use `k_values` for the x-axis and the lists of train and test  $R^2$  scores as y-axis values
  1. To create the plot, you need to set up the Figure and Axes objects using `plt.subplots()`.
    - These two objects (fig and ax) will allow you to control various properties of the figure and plot.
    - `fig` is the Figure object: It serves as the overall container for the plot.
    - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.

Example code: `fig, ax = plt.subplots(figsize=(10, 6))`
  2. Customize the Plot:
    - You need to set the title of the plot, the labels for the x-axis and y-axis, and format the ticks on the x-axis for better readability.

```
[ ]: """Write your code for exercise-12 here:"""
```

```
# your code here  
raise NotImplementedError
```

```
[ ]:
```

**2 END**